

Operating System

Chapter 11: File System Implementation

Student: Nguyễn Đức Mạnh

Student ID: HE204816

Mục lục

1	11.1 RAM Disk and Disk Cache	2
2	11.2 Contiguous Allocation with Extents	2
3	11.3 Allocation Granularity: 4KB and 512 Bytes	3
4	11.4 Advantages of FAT	3
5	11.5 I/O Operations for a 100-block File	3
6	11.6 Maximum File Size with Inode	4

11.1 RAM Disk and Disk Cache

Disk Cache

Disk Cache is a part of main memory used by the operating system to store recently accessed disk blocks. It is managed automatically by the operating system, usually using replacement algorithms such as LRU. Data in disk cache can be removed at any time when memory is needed for other purposes.

RAM Disk

RAM Disk is a region of RAM that the operating system presents to the user as if it were a physical disk drive.

Useful situations

- **Data control:** When users want to ensure that some temporary files, such as compiler files `.obj` or `.tmp`, always stay in memory for the fastest access speed.
- **Diskless devices:** RAM Disk is useful for diskless workstations where temporary file systems must be stored entirely in RAM.

11.2 Contiguous Allocation with Extents

This mechanism improves pure contiguous allocation by allowing a file to consist of several contiguous extents instead of only one contiguous block.

a. Fixed-size extents

- Free-space management becomes simple, similar to paging.
- The system only needs to manage blocks of equal size.
- However, if a file needs only 1KB but the fixed extent size is 64KB, serious **internal fragmentation** occurs.

b. Variable-size extents

- Files can be allocated more accurately according to their real size.
- This reduces internal waste.
- However, after many files are created and deleted, free spaces become scattered, causing **external fragmentation**.

c. Several fixed sizes

- For example, the system may provide extents of 4KB, 64KB, and 1MB.
- This is a balanced solution.

- It reduces search complexity compared with variable-size allocation and reduces internal fragmentation compared with using only one fixed size.

11.3 Allocation Granularity: 4KB and 512 Bytes

Main idea

A file system can combine large blocks and small blocks to improve both performance and storage efficiency.

- **Large blocks, 4KB:** Used for large data files to reduce the number of index entries and improve I/O throughput.
- **Small blocks, 512 bytes:** Used for small files or the remaining part at the end of large files to reduce internal fragmentation.
- **Free-space management:** The bitmap must track space at the 512-byte level. One 4KB block corresponds to 8 bits in the bitmap.

11.4 Advantages of FAT

Random Access

FAT improves linked allocation significantly. Instead of reading each disk block to find the pointer to the next block, the system only needs to look up the **File Allocation Table**. Because FAT is usually cached in main memory, finding any block becomes much faster.

Useful Capacity

In normal linked allocation, each block must reserve some bytes for a pointer. With FAT, the whole disk block can be used for data because pointers are stored separately in the FAT table.

11.5 I/O Operations for a 100-block File

Assume that management blocks such as FCB or Index Block are already in RAM.

Operation		Contiguous	Linked	Indexed
Insert at beginning		Move 100 blocks: read 100 + write 1 new block = 201	Write new block and update pointer in FCB = 1	Write new block + rewrite index block = 2

Insert in middle, block 50	Move 50 blocks: read 50 + write 50 + write 1 new block = 101	Find block 49: read 50 + write new block 50 + rewrite block 49 = 52	Write new block + rewrite index block = 2
Insert at end	Write block 101 = 1	Write block 101 + rewrite block 100 = 2	Write block 101 + rewrite index block = 2
Delete in middle, block 50	Move 50 blocks: read 50 + write 50 = 100	Read 50 blocks to find it + rewrite block 49 = 51	Rewrite index block only = 1

11.6 Maximum File Size with Inode

Given information

Block size: $8KB = 2^{13}$ bytes
Pointer size: 4 bytes

Number of pointers per block

$$\frac{8192}{4} = 2048 = 2^{11}$$

Calculation

- Direct blocks:

$$12 \times 8KB = 96KB$$

- Single indirect block:

$$2048 \times 8KB = 16MB$$

- Double indirect block:

$$2048 \times 2048 \times 8KB = 32GB$$

- Triple indirect block:

$$2048 \times 2048 \times 2048 \times 8KB = 2^{11} \times 2^{11} \times 2^{11} \times 2^{13} = 2^{46} \text{ bytes} = 64TB$$

Conclusion

The maximum file size is:

$$96KB + 16MB + 32GB + 64TB \approx 64TB$$